

Evasion Resistance in Network Intrusion Detection: A Comparative Study of Rule-Based, CNN-Based, and Random Forest Approaches

Katayoun Rafat
University of Dubai
Dubai, UAE
s000003693@ud.ac.ae

Abstract—

[TODO:] Write last. 150–250 words. Should state: (1) the problem – most IDS comparisons report clean-data accuracy only, ignoring evasion resistance; (2) what you did – compared a rule-based IDS (NetSentry), a 1D-CNN, and a Random Forest on CICIDS2017, evaluating both clean-data performance and evasion susceptibility; (3) key finding – CNN and RF achieve near-identical clean-data performance (99.7–99.96% accuracy), which reframes the comparison toward robustness rather than raw accuracy; NetSentry’s per-source-IP volumetric rule, approximated at flow granularity, fails to detect 100% of flow-level DoS attacks in the tested dataset while still producing false positives on benign traffic; (4) implication – architectural complexity (CNN) does not by itself confer evasion robustness over simpler ML (RF), and rule-based systems relying on aggregate/volumetric signals are structurally blind when attack traffic is deliberately kept low-and-slow per connection.

Index Terms—intrusion detection, convolutional neural network, random forest, adversarial evasion, CICIDS2017, network security

I. INTRODUCTION

Network intrusion detection systems (IDS) generally follow one of two design philosophies. Rule-based systems apply fixed, human-defined thresholds and signatures to traffic – fast, interpretable, and inexpensive to run, but reliant on thresholds that must be manually tuned and that attackers can potentially probe and evade once discovered. Machine learning-based systems, ranging from classical ensemble methods to deep neural networks, learn detection boundaries directly from labeled traffic data – often achieving very high accuracy on held-out test data, at the cost of interpretability, training cost, and an implicit assumption that future attack traffic resembles the traffic seen during training.

Published comparisons between these paradigms, and between different machine learning architectures within the second category, overwhelmingly report accuracy, precision, recall, or F1-score on a single, in-distribution test split. This is a reasonable starting point, but it leaves an important question unanswered: intrusion detection is fundamentally an adversarial setting, where a capable attacker actively seeks to evade whatever detection logic is deployed. A method that performs well on clean, held-out data drawn from the same distribution as its training data offers no guarantee of similar performance against traffic deliberately crafted to evade it, or even against traffic from an attack category the method

was never exposed to during training. Evasion resistance and cross-attack generalization, not clean-data accuracy alone, are the properties that matter for real-world deployment – yet they remain comparatively under-studied relative to accuracy benchmarking.

This paper is motivated by a specific empirical observation that emerged during the course of this work: on the CICIDS2017 dataset, a Random Forest classifier matches or marginally exceeds a 1D convolutional neural network (CNN) on every clean-data metric – accuracy, precision, recall, F1, and ROC-AUC – while training in roughly one-twentieth the wall-clock time, on identical hardware, with no GPU required by either method (Section III-F). If a substantially cheaper classical model matches a deep learning model’s performance on clean data, clean-data accuracy alone cannot justify the additional complexity and cost of the deep learning approach. This raises the paper’s central question: does the CNN’s architectural complexity confer any advantage over the simpler Random Forest once the comparison moves beyond clean, in-distribution data – specifically, under adversarial evasion or when tested against an attack category outside the training distribution?

To situate this question against a real deployed system rather than only comparing two machine learning methods to each other, this paper additionally evaluates NetSentry, a rule-based IDS with production-style detection thresholds (Section III-C), on the same data and under the same evaluation protocol. This three-way comparison surfaces a further methodological contribution: CICIDS2017’s flow-level data format does not retain the per-source-IP information NetSentry’s volumetric rules require, forcing an explicit, documented approximation whose consequences are analyzed directly (Sections IV-B and V) rather than glossed over.

The contributions of this paper are:

- A fair, identically-partitioned three-way comparison of a production-style rule-based IDS (NetSentry), a CNN, and a Random Forest on CICIDS2017, evaluated under both clean-data conditions and cross-attack-type generalization (Wednesday DoS traffic and Friday port-scan traffic).
- A documented methodology for evaluating a per-source-IP rule-based system on flow-level data lacking source-IP identifiers, including an explicit accounting of what de-

tection capability is and is not preserved by the resulting approximation.

- An empirical adversarial evasion analysis (Section VI) informed directly by Random Forest feature importance, testing whether crafted traffic perturbations can evade one detection method while remaining detectable by another.
- A practical discussion of what these results imply for practitioners choosing between rule-based and machine learning-based detection paradigms, and between machine learning architectures of differing complexity.

II. RELATED WORK

A. Rule-Based and Signature-Based IDS

Traditional intrusion detection systems rely on fixed, human-defined signatures or thresholds compared against observed traffic. This approach is computationally inexpensive and produces interpretable, auditable decisions – an alert can always be traced back to the specific rule that fired – but it depends on manually tuned thresholds that must be set correctly for the traffic patterns and attack tools the defender expects to see, and it cannot detect attack patterns that do not match a predefined signature. Once an attacker learns or infers a system’s threshold values, evasion becomes a matter of staying below them. NetSentry, evaluated in this paper, follows this design philosophy directly, with detection logic based on fixed packet-count and port-based thresholds (Section III-C).

B. Machine Learning and Deep Learning IDS on CICIDS2017

CICIDS2017 [1] has become a standard benchmark for evaluating machine learning-based intrusion detection, and numerous studies report high accuracy for both classical and deep learning models on it. Xu and Liu [2] present a controlled comparison of MLP, 1D CNN, One-Class SVM, and Local Outlier Factor on CICIDS2017 under two conditions: detecting known attack types and generalizing to previously unseen threats – a structure closely aligned with the two-part evaluation in this paper (Sections III-F and IV-C). Ali et al. [3] directly compare Random Forest against CNN and LSTM on CICIDS2017 and report Random Forest achieving the highest accuracy (99.9%) among all models tested, with CNN and LSTM both reaching approximately 98%. This finding is consistent with the clean-data parity between Random Forest and CNN observed independently in this paper (Section III-F), reported across a different train/test split, feature set, and hyperparameter configuration – suggesting the pattern is not an artifact of one particular experimental setup. Despite this recurring observation across independent studies, the great majority of this literature, including the two studies above with the exception of Xu and Liu’s generalization experiments, evaluates models exclusively on in-distribution, same-attack-type held-out data, leaving the evasion-resistance and cross-attack-generalization questions this paper focuses on comparatively underexplored.

C. Adversarial Evasion of Flow-Based ML-NIDS

A separate line of work examines the vulnerability of ML-based NIDS to adversarial evasion. elShehaby and Matrawy [4] introduce the concept of feature *perturb-ability* for flow-based ML-NIDS: a metric quantifying how susceptible a given flow feature is to attacker manipulation in the problem-space (i.e., via actually altered network traffic) without breaking the underlying attack’s functionality or producing invalid traffic. This directly motivates the evasion strategy explored in Section VI of this paper: the packet-length features identified as most important to the Random Forest’s decisions (Section V) are plausible candidates for problem-space perturbation, since padding packet size is achievable by an attacker without necessarily compromising the attack’s effect, unlike features more tightly constrained by protocol semantics. Where prior work in this space generally evaluates evasion against machine learning models in isolation, this paper extends the comparison to include a production-style rule-based system evaluated under the same perturbation strategy, testing whether a given evasion technique transfers across fundamentally different detection paradigms rather than only across different ML architectures.

III. METHODOLOGY

A. Dataset

We use the CICIDS2017 dataset [1] (Canadian Institute for Cybersecurity Intrusion Detection Evaluation Dataset, 2017), a widely-used labeled network traffic dataset combining benign and attack traffic captured over five days. This study uses two subsets: the Wednesday-workingHours capture, which contains denial-of-service (DoS) attack traffic (DoS Hulk, DoS GoldenEye, DoS Slowloris, DoS Slowhttptest, and Heartbleed), and the Friday-Afternoon-PortScan capture, used to evaluate generalization to a structurally different attack class. Traffic is pre-extracted into flow-level records using CICFlowMeter, yielding 78 statistical features per flow (e.g., packet counts, flow duration, inter-arrival times, packet length statistics). This flow-level granularity, rather than raw packet capture, is an important methodological detail: it later constrains how rule-based detection logic originally designed for per-packet, per-source-IP monitoring can be evaluated on this dataset (Section III-C).

Heartbleed traffic was excluded from the Wednesday subset, as it comprises only 11 flows – insufficient for stable evaluation of any detection method. The resulting cleaned Wednesday dataset contains 691,395 flows.

B. Preprocessing

Column names and string values were stripped of leading/trailing whitespace, a known artifact of the original CICIDS2017 CSV export. Rows containing infinite or missing values in numeric features – a known issue in CICIDS2017 arising from division-by-zero in rate-based features (e.g., Flow Bytes/s) when flow duration is zero – were removed during preprocessing; the resulting cleaned dataset contains no remaining NaN or infinite values in any numeric column, verified directly against the final cleaned file.

Labels were converted to a binary classification target: BENIGN mapped to 0, and all attack labels mapped to 1. The dataset was split into training and test sets using stratified sampling (80%/20%) with a fixed random seed (`random_state=42`) to preserve class balance across both splits and ensure the same partition could be exactly reproduced for the rule-based baseline evaluation (Section III-C).

Nine zero-variance columns (features with a single constant value across all training rows) were identified on the training split and dropped from both splits, reducing the feature count from 78 to 68. A `StandardScaler` was fit exclusively on the training split and applied to transform both training and test splits, preventing information leakage from the test set into the scaling parameters.

C. Rule-Based Baseline: NetSentry

NetSentry is a real-time, rule-based intrusion detection system implemented with FastAPI and Scapy, with JWT/RBAC authentication and encrypted alert storage. This paper evaluates only its packet/flow-level detection logic; the authentication and storage layers are not relevant to detection accuracy and are omitted from further discussion.

Real thresholds (from `detection.py`):

- DOS_ATTEMPT: ≥ 50 packets from one source IP within a 10-second window.
- PORT_SCAN: > 8 distinct destination ports contacted by one source IP.
- SUSPICIOUS_PORT: $\text{destination port} \in \{4444, 31337, 12345, 6667, 1337\}$.

Critical methodological note: CICIDS2017’s ML-ready flow CSVs are generated by CICFlowMeter at per-connection (flow) granularity and do not retain per-source-IP identifiers needed to reconstruct NetSentry’s true aggregate rule. We therefore approximate DOS_ATTEMPT at flow level:

- For flows with duration ≤ 10 s: use raw total packet count directly (no extrapolation, since a short window cannot contain more packets than the flow actually carried).
- For flows with duration > 10 s: estimate the densest possible 10-second slice via the flow’s average packet rate, i.e. $\text{packets_per_10s} = (\text{total_packets}/\text{duration_seconds}) \times 10$.

Flow Duration is recorded in microseconds in CICIDS2017 and is converted to seconds before use. SUSPICIOUS_PORT is applied exactly as specified, since it requires only the Destination Port field. PORT_SCAN requires grouping by source IP across many flows and is evaluated separately using the Friday dataset.

The full PORT_SCAN rule (which requires counting distinct destination ports contacted by a single source IP) could not be evaluated for the same reason as DOS_ATTEMPT: CICIDS2017’s flow-level records lack the source-IP field needed to reconstruct this aggregate behavior. The SUSPICIOUS_PORT rule, by contrast, requires no cross-flow aggregation – it is a per-flow check – and was evaluated exactly, without approximation, on the Friday-Afternoon-PortScan dataset (Section IV).

This is an approximation of a per-source-IP rule at flow granularity, and the Results/Discussion sections show this has material consequences for detection performance.

D. CNN Architecture

The CNN is a compact 1D convolutional architecture operating over the 68-feature flow vector, treated as a single-channel sequence:

- Conv1D (32 filters, kernel size 3, ReLU) $\rightarrow$ MaxPooling1D (pool size 2)
- Conv1D (64 filters, kernel size 3, ReLU) $\rightarrow$ MaxPooling1D (pool size 2)
- Dense (64 units, ReLU) $\rightarrow$ Dropout (rate 0.3)
- Dense (1 unit, sigmoid) – binary output (0 = benign, 1 = attack)

The model was compiled with the Adam optimizer and binary cross-entropy loss, appropriate for this binary classification task. Training used a 90/10 train/validation split (10% of the training data held out to monitor overfitting), 10 epochs, and a batch size of 256.

E. Random Forest

The Random Forest baseline uses scikit-learn’s `RandomForestClassifier` with 100 trees (`n_estimators=100`) and unrestricted tree depth (`max_depth=None`, trees grown until leaves are pure or contain fewer than the minimum split threshold). Class weighting was not explicitly applied despite the benign/attack class imbalance present in the dataset; this is a deliberate baseline configuration rather than an oversight, and its effect on the reported metrics is discussed in Section VII. All experiments, including both CNN and Random Forest training, were run on a single machine (Intel Core i7-1360P CPU, no GPU acceleration – TensorFlow GPU support is unavailable on native Windows for the version used, and training proceeded on CPU only for both models, keeping hardware constant across the comparison). Random Forest training completed in 19.12 seconds (logged wall-clock time), compared to on the order of several minutes for the CNN’s 10 training epochs – an approximately order-of-magnitude difference despite both running on identical hardware. Random Forest inference over the full test set (138,279 flows) took 0.29 seconds.

F. Evaluation Protocol

The three detection methods compared in this paper represent three distinct design points on the interpretability-complexity spectrum: NetSentry represents the rule-based paradigm still common in deployed network security tooling; Random Forest represents a classical ensemble machine learning approach with moderate training cost and built-in feature importance; and the CNN represents a deep learning approach capable of learning feature interactions automatically, at higher training cost. Comparing all three under an identical evaluation protocol isolates the effect of detection paradigm from confounds such as differing test sets or preprocessing.

All three detection methods – NetSentry, CNN, and Random Forest – are evaluated on the identical held-out test split described in Section III-C, ensuring a fair, directly comparable evaluation. Because the NetSentry rule-based baseline requires re-deriving the same train/test partition independently, an explicit sanity check was performed: the recovered test-set labels were asserted to exactly match the labels saved during the original CNN/RF split, confirming row-for-row equivalence before any comparison was made.

We report four standard classification metrics for all three methods: *accuracy* (the proportion of all flows correctly classified), *precision* (the proportion of flows flagged as attacks that are true attacks), *recall* (the proportion of true attacks successfully flagged), and *F1-score* (the harmonic mean of precision and recall). We additionally report ROC-AUC for the CNN and Random Forest, which summarizes classification performance across all possible decision thresholds. NetSentry, as a deterministic binary rule engine, produces no probability score, so ROC-AUC is not applicable to it – this is stated explicitly rather than omitted, since naively comparing AUC only across the two ML methods could understate this as a limitation of the comparison itself rather than a property of NetSentry’s design. CNN stability was additionally verified via 5-fold stratified cross-validation.

[TODO:] Fix cross-reference above once needed – currently points to netsentry-baseline for the split description, but the split is actually defined in Preprocessing. Consider adding a \label to the Preprocessing subsection and pointing here instead.

IV. RESULTS

A. Detection Performance on Wednesday (DoS) Test Set

Table I gives a high-level summary of the three methods before detailed metrics follow in Table II.

TABLE I
HIGH-LEVEL COMPARISON OF THE THREE DETECTION METHODS.

Model	DoS Recall	Train Time	Cross-Attack Recall
NetSentry	0.00%	N/A (rule-based)	0.40%
CNN	99.99%	Several minutes	0.13%
Random Forest	99.98%	19.12s	0.09%

Table II reports accuracy, precision, recall, and F1-score for all three methods on the held-out Wednesday DoS test set, using the identical partition described in Section III-F.

TABLE II
DETECTION PERFORMANCE ON THE WEDNESDAY (DoS) TEST SET. PRECISION/RECALL/F1 ARE REPORTED FOR THE ATTACK CLASS.

Model	Acc.	Prec.	Rec.	F1
NetSentry (flow-approx.)	0.6295	0.0000	0.0000	0.0000
CNN	0.9970	0.9921	0.9999	0.9959
Random Forest	0.9996	0.9990	0.9998	0.9994

Random Forest achieves marginally better precision, recall, and F1 than the CNN despite requiring no GPU and a fraction of the training time (Section III-F).

The NetSentry rule, approximated at flow granularity, achieves 0% recall on the ATTACK class: none of the 50,342 attack flows in the test set were flagged (0 true positives, 50,342 false negatives), while still producing 886 false positives on benign traffic. The CNN correctly identifies 99.99% of attacks (50,335 true positives, 7 false negatives out of 50,342), at the cost of 403 false positives on benign traffic (precision 0.9921). Random Forest achieves comparable or slightly better performance (99.96% accuracy, ROC-AUC 1.0000) at roughly an order of magnitude less training time (Section III-F).

CNN stability across data splits was confirmed via 5-fold stratified cross-validation: accuracy $99.71\% \pm 0.02\%$, recall $99.96\% \pm 0.03\%$ (cv_results.csv).

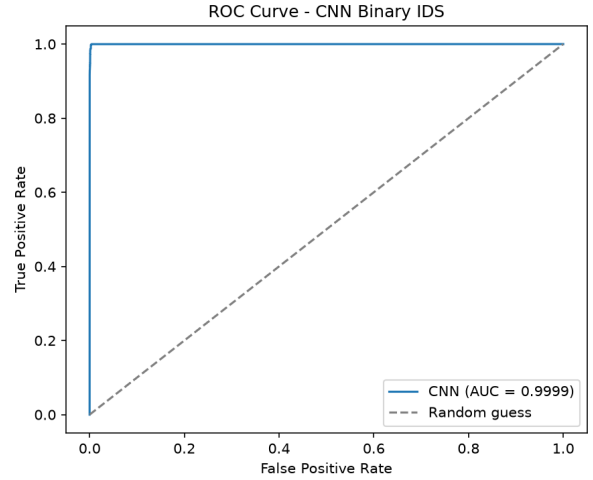


Fig. 1. ROC curve for the CNN on the Wednesday (DoS) test set (AUC = 0.9999).

B. Diagnosing NetSentry’s Zero Recall

To understand why the flow-level DOS_ATTEMPT approximation detects no attacks at all, we computed the packets-per-10-second-window value (Section III-C) directly for every flow in the test set, grouped by true label. Table III summarizes the maximum value observed for each DoS attack subtype, against NetSentry’s real threshold of 50.

TABLE III
MAXIMUM OBSERVED PACKETS-PER-10S VALUE BY ATTACK TYPE, AGAINST THE 50-PACKET DOS_ATTEMPT THRESHOLD.

Label	Max packets/10s
DoS Hulk	27
DoS GoldenEye	21
DoS Slowloris	11
DoS Slowhttptest	10
NetSentry threshold	50

No attack subtype approaches the 50-packet threshold at any point in the test set – even DoS Hulk, typically considered a high-volume flooding tool, never exceeds 27 packets within any 10-second-equivalent window at the single-flow level.

C. Generalization to a Structurally Different Attack Type: Port Scanning

To test whether detection performance generalizes beyond the attack category used for training, we evaluated all three methods on the Friday-Afternoon-PortScan capture – a structurally different attack (reconnaissance via many distinct destination ports) that neither the CNN nor Random Forest was trained on. Table IV reports results for the CNN and RF (both trained exclusively on Wednesday DoS traffic) and for NetSentry’s SUSPICIOUS_PORT rule, evaluated exactly (no flow-level approximation required, since it is a per-flow check against a fixed port set). NetSentry’s true PORT_SCAN rule (distinct-port count per source IP) could not be evaluated for the same source-IP-aggregation reason discussed in Section III-C.

TABLE IV
GENERALIZATION TO FRIDAY PORT-SCAN TRAFFIC. CNN AND RF WERE TRAINED ONLY ON WEDNESDAY DOS DATA.

Model	Acc.	Prec.	Rec.	AUC
CNN (DoS → PortScan)	0.4422	0.1727	0.0013	0.5970
RF (DoS → PortScan)	0.4452	0.6938	0.0009	0.7869
NetSentry SUSPICIOUS_PORT	0.4471	0.9985	0.0040	n/a

All three methods fail to detect the large majority of port-scan traffic: recall is below 0.5% in every case, meaning each method misses more than 99.5% of scanning flows when it has not been trained or designed for this specific attack category.

Random Forest’s ROC-AUC on port-scan data is 0.7869, compared to 0.5970 for the CNN; NetSentry’s SUSPICIOUS_PORT rule achieves the highest precision (0.9985) among the three methods but the lowest recall (0.0040).

V. DISCUSSION

A. Does CNN Complexity Buy Anything Over a Simpler Model?

Table II shows Random Forest matching or marginally exceeding the CNN on every clean-data metric (accuracy, precision, recall, F1, ROC-AUC), while training in 19.12 seconds versus on the order of several minutes for the CNN – roughly an order of magnitude faster, on identical hardware, with no GPU required by either method. On clean, in-distribution data, the CNN’s additional architectural complexity produces no measurable benefit over a substantially cheaper classical ensemble method. This result reframes the central question of this paper: if raw accuracy does not distinguish these two approaches, the deciding factor for practitioners must be sought elsewhere – specifically, in behavior under conditions the clean test set does not capture, such as adversarial evasion

(Section VI) or generalization to unseen attack types (Section IV-C).

B. Interpreting NetSentry’s Zero Recall

The 0% recall reported for NetSentry’s flow-level DOS_ATTEMPT approximation (Section IV-B) is explained directly by the packet-rate diagnostic in Table III: no DoS attack subtype in the test set ever approaches the 50-packet threshold at the single-flow level, even DoS Hulk, typically considered a high-volume flooding tool. This is consistent with CICIDS2017’s flow generation methodology: CICFlowMeter records each TCP/UDP connection as an individual flow, and a volumetric DoS attack achieves its effect through many simultaneous low-to-moderate-rate connections rather than one connection carrying an extreme packet rate. NetSentry’s real rule aggregates packet counts *across* flows sharing a source IP; a single-flow approximation structurally cannot see this aggregate behavior, so the 0% recall reflects a granularity limitation in the evaluation methodology rather than a flaw in the 50-packet threshold value itself.

This result should not be read as a blanket indictment of the deployed system. NetSentry’s real rule aggregates packet counts across all flows sharing a source IP; the version evaluated here is necessarily a single-flow approximation, forced by CICIDS2017’s lack of per-source-IP identifiers in its ML-ready format (Section III-C). The deployed system, operating on live packet captures with full IP visibility, may perform considerably better against the same underlying attack traffic.

What this result *does* demonstrate is a specific, generalizable vulnerability: detection logic based on a per-connection volumetric threshold is structurally blind to attacks distributed across many parallel, individually low-rate connections. This is not merely an artifact of the evaluation methodology – it describes a real evasion strategy available to an attacker against the deployed per-source-IP rule as well, achieved simply by distributing attack traffic across many source IPs or throttling the rate per connection below the aggregate threshold. In this sense, the flow-level approximation used here inadvertently previews a class of evasion technique relevant to the full system, motivating the adversarial evasion experiments in Section VI.

C. Threshold Calibration and Cross-Attack Generalization

The Friday port-scan generalization results (Section IV-C) show all three methods failing to detect an attack type outside their training distribution or design intent, with recall below 0.5% in every case – a substantially stronger and more general finding than the DoS-specific result in Section IV-B, since it demonstrates that neither architectural complexity (CNN) nor a simpler ensemble method (RF) provides meaningful cross-attack-type generalization on its own. NetSentry’s SUSPICIOUS_PORT rule achieves very high precision (99.85%) alongside its low recall, consistent with its design intent of flagging known malicious ports rather than detecting scanning behavior in general; its near-total failure to catch port scans reflects that generic scanning tools do not typically target this

fixed port set, not a defect in the rule’s logic for its intended purpose, and is not a substitute for true distinct-port-count scan detection. However, Random Forest’s ROC-AUC on this out-of-distribution data (0.7869) is well above chance, despite its near-zero recall at the default 0.5 decision threshold. This gap between a moderately informative ranking (AUC) and a near-total failure to classify (recall) indicates the model has *not* lost all ability to distinguish port-scan traffic from benign traffic – it has retained partial discriminative signal, but the decision threshold learned from DoS training data does not transfer to this structurally different attack type. This suggests that per-deployment or per-attack-type threshold recalibration, rather than full retraining, may recover meaningful detection capability in some cross-attack-type scenarios – a considerably cheaper intervention than collecting labeled data for every new attack category. Whether this holds in practice is a natural direction for future work beyond the scope of this paper.

D. Implications for Evasion Testing

Feature importance analysis of the Random Forest model identifies packet-length-related statistics as dominant in the ATTACK/BENIGN decision: 7 of the top 10 features by importance are packet-length metrics, led by Backward Packet Length Mean (importance 0.0965), followed by Packet Length Mean (0.0670), Backward Packet Length Std (0.0571), Average Packet Size (0.0568), Backward Packet Length Max (0.0496), Max Packet Length (0.0431), and Packet Length Variance (0.0418) – together accounting for over a third of total feature importance. Destination Port (0.0529) is the only non-length feature in the top five, notable given NetSentry’s own SUSPICIOUS_PORT rule also relies on destination port as a signal, via a very different mechanism (a fixed known-malicious set rather than a learned statistical association).

This concentration of importance in packet-length statistics motivates a concrete evasion hypothesis for Section VI: an attacker padding packet sizes to shift these features toward values typical of benign traffic may be able to reduce detection confidence in the ML-based methods without materially changing the attack’s functional effect. Whether this evasion strategy succeeds against the CNN, Random Forest, both, or neither is evaluated directly in the adversarial evasion experiments that follow.

VI. ADVERSARIAL EVASION TESTING

[TODO:] PHASE 4 – NOT STARTED. *This is the paper’s core contribution. Outline once designed:*

- *Craft traffic perturbations informed by feature importance (e.g., pad packet lengths, adjust inter-arrival timing, split volume across more/fewer flows).*
- *Test whether crafted samples evade NetSentry but not CNN/RF, evade CNN but not RF, or evade all three.*
- *Report evasion success rate per model, per perturbation strategy.*

VII. LIMITATIONS

[TODO:]

- *NetSentry’s DOS_ATTEMPT rule could only be approximated at flow level due to CICIDS2017 lacking per-source-IP grouping in the ML-ready CSV; true per-IP aggregate behavior may perform differently on live traffic.*
- *Single dataset (CICIDS2017) – generalization to other traffic distributions/attack tools untested.*
- *Add more once Phase 4/5 are done – e.g. seed sensitivity, dataset age (CICIDS2017 is from 2017; attack tooling has evolved).*

VIII. CONCLUSION

[TODO:] *Write last, 1 paragraph. Restate central finding and practical takeaway for practitioners choosing between IDS paradigms.*

ACKNOWLEDGMENT

[TODO:] *optional*

REFERENCES

- [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [2] Z. Xu and Y. Liu, “Robust anomaly detection in network traffic: Evaluating machine learning models on cids2017,” *arXiv preprint arXiv:2506.19877*, 2025.
- [3] M. L. Ali, K. Thakur, S. Schmeelk, J. DeBello, and D. Dragos, “Deep learning vs. machine learning for intrusion detection in computer networks: A comparative study,” *Applied Sciences*, vol. 15, no. 4, p. 1903, 2025.
- [4] M. elShehaby and A. Matrawy, “A novel perturb-ability score to mitigate evasion adversarial attacks on flow-based ml-nids,” *arXiv preprint arXiv:2409.07448*, 2024.